



E-commerce Business Analysis using SQL & Python

From Data to Insights

Nivas Kumar

BASIC PROBLEMS

1. List all unique cities where customers are located.
2. Count the number of orders placed in 2017.
3. Find the total sales per category.
4. Calculate the percentage of orders that were paid in installments.
5. Count the number of customers from each state.

1. List all unique cities where customers are located.

```
unique_cities_df = (  
    customers[['customer_city']]  
    .dropna()  
    .drop_duplicates()  
    .sort_values(by='customer_city')  
    .reset_index(drop=True)  
)  
  
display(unique_cities_df)  
  
print("Total unique cities:", unique_cities_df.shape[0])  
[15] ✓ 0.0s
```

	customer_city
0	abadia dos dourados
1	abadiania
2	abaete
3	abaetetuba
4	abaiara
...	...
4114	xinguara
4115	xique-xique
4116	zacarias
4117	ze doca
4118	zortea

4119 rows × 1 columns

The platform operates at a national scale with strong geographic reach, covering over 4119 cities across the nation, suggesting mature logistics infrastructure and diverse regional demand patterns.

```
2 • SELECT DISTINCT  
3     customer_city  
4 FROM customers  
5 ORDER BY customer_city;
```

customer_city
▶ abadia dos dourados
abadiania
abaete
abaetetuba
abaiara
abaira
abare
abatia
abdon batista
abelardo luz
abranes
abre campo
abreu e lima
acaiaca
acailandia
acajutiba
acarau
acari
acegua
acopiara
acreuna

2. Count the number of orders placed in 2017

```
# Ensure timestamp column is datetime
orders['order_purchase_timestamp'] = pd.to_datetime(orders['order_purchase_timestamp'])

# Filter orders from 2017
orders_2017 = orders[orders['order_purchase_timestamp'].dt.year == 2017]

# Count distinct orders
total_orders_2017 = orders_2017['order_id'].nunique()

print("Total orders in 2017:", total_orders_2017)
```

[16] ✓ 0.2s

... Total orders in 2017: 45101

```
7  -- Basic Problem 2: Count number of orders placed in 2017
8  •  SELECT
9      COUNT(DISTINCT order_id) AS total_orders_2017
10 FROM orders
11 WHERE YEAR(order_purchase_timestamp) = 2017;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	total_orders_2017
▶	45101

The resulted number 45101, establishes the baseline operational scale of the business.

When we compute:

- Revenue
- YoY growth
- Retention rate

We will use 2017 as a reference benchmark year.

3. Find the total sales per product category

	product_category	total_sales
30	HEALTH BEAUTY	1258681.34
45	Watches present	1205005.68
49	bed table bath	1036988.68
68	sport leisure	988048.97
53	computer accessories	911954.32
...	...	...
58	flowers	1110.04
32	House Comfort 2	760.27
50	cds music dvds	730.00
18	Fashion Children's Clothing	569.85
62	insurance and services	283.29

73 rows × 2 columns

Overall revenue: 13591643.7

Key Insights:

- Revenue is **highly concentrated** in top categories.
- **Health & Beauty** is the strongest revenue driver.
- Home & Lifestyle categories show strong demand.
- Long-tail effect: 73 categories, but few dominate revenue.
- Business performance depends heavily on top 5–10 categories.

```
13 -- Basic Problem 3: Total sales per product category
14 • SELECT
15     p.product_category,
16     ROUND(SUM(oi.price), 2) AS total_sales
17 FROM order_items oi
18 JOIN products p
19     ON oi.product_id = p.product_id
20 GROUP BY p.product_category
21 ORDER BY total_sales DESC;
22
```

product_category	total_sales
HEALTH BEAUTY	1258681.34
Watches present	1205005.68
bed table bath	1036988.68
sport leisure	988048.97
computer accessories	911954.32
Furniture Decoration	729762.49
Cool Stuff	635290.85
housewares	632248.66
automotive	592720.11
Garden tools	485256.46
toys	483946.60
babies	411764.89
perfumery	399124.87
telephony	323667.53
Furniture office	273960.70

```
23 -- Checking total sales
24 • SELECT
25     ROUND(SUM(price), 2) AS overall_revenue
26 FROM order_items;
27
28
```

overall_revenue
13591643.70

4. Calculate the percentage of orders that were paid in installments

```
# Total distinct orders
total_orders = orders['order_id'].nunique()

# Orders paid in installments (>1)
installment_orders = (
    payments[payments['payment_installments'] > 1]['order_id']
    .nunique()
)

# Calculate percentage
installment_percentage = round(
    (installment_orders / total_orders) * 100,
    2
)

print("Total Orders:", total_orders)
print("Installment Orders:", installment_orders)
print("Installment Percentage:", installment_percentage, "%")
```

✓ 0.0s

Total Orders: 99441
Installment Orders: 51170
Installment Percentage: 51.46 %

```
28 -- Basic Problem 4: Percentage of orders paid in installments
29 • SELECT
30     ROUND(
31     (
32         SELECT COUNT(DISTINCT order_id)
33         FROM payments
34         WHERE payment_installments > 1
35     )
36     /
37     (
38         SELECT COUNT(DISTINCT order_id)
39         FROM orders
40     ) * 100,
41     2
42 ) AS installment_percentage;
43
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	installment_percentage			
▶	51.46			

Installment Payment Behavior

- **51.46%** of all orders were paid in installments.
- More than **half of customers prefer installment payments**.
- Indicates strong dependency on flexible payment options.
- Installment financing likely drives higher order conversion.
- Removing installment options could significantly reduce sales.

5. Count the number of customers from each state

	customer_state	total_customers
25	SP	40302
18	RJ	12384
10	MG	11259
22	RS	5277
17	PR	4882
23	SC	3534
4	BA	3277
6	DF	2075
7	ES	1964
8	GO	1952
15	PE	1609
5	CE	1313
13	PA	949
12	MT	876
9	MA	726
11	MS	694
14	PB	519
16	PI	482
19	RN	474
1	AL	401
24	SE	342
26	TO	273
20	RO	240
2	AM	143
0	AC	77
3	AP	67
21	RR	45

Key Insights

- **Sao Paulo(SP) dominates**, contributing ~40% of total customers.
- Customer base is heavily concentrated in Southeast Brazil.
- Strong regional demand concentration risk.
- Northern states contribute significantly less customer volume.
- Geographic expansion opportunity exists in underrepresented regions.

```
45 -- Basic Problem 5: Count number of unique customers from each state
46 • SELECT
47     customer_state,
48     COUNT(DISTINCT customer_unique_id) AS total_customers
49 FROM customers
50 GROUP BY customer_state
51 ORDER BY total_customers DESC;
```

customer_state	total_customers
SP	40302
RJ	12384
MG	11259
RS	5277
PR	4882
SC	3534
BA	3277
DF	2075
ES	1964
GO	1952
PE	1609
CE	1313
PA	949
MT	876
MA	726
MS	694
PB	519
PI	482
RN	474
AL	401
SE	342
TO	273
RO	240
AM	143
AC	77
AP	67
RR	45

INTERMEDIATE PROBLEMS

1. Calculate the number of orders per month in 2018.
 2. Find the average number of products per order, grouped by customer city.
 3. Calculate the percentage of total revenue contributed by each product category.
 4. Identify the correlation between product price and the number of times a product has been purchased.
 5. Calculate the total revenue generated by each seller, and rank them by revenue.
- 

1. Calculate the number of orders per month in 2018

```
53 -- Intermediate Problem 1: Number of orders per month in 2018
54 • SELECT
55     MONTH(order_purchase_timestamp) AS order_month,
56     COUNT(DISTINCT order_id) AS total_orders
57 FROM orders
58 WHERE YEAR(order_purchase_timestamp) = 2018
59 GROUP BY order_month
60 ORDER BY order_month;
61
62 -- Verifying Months
63 • SELECT
64     MIN(order_purchase_timestamp) AS earliest_2018,
65     MAX(order_purchase_timestamp) AS latest_2018
66 FROM orders
67 WHERE YEAR(order_purchase_timestamp) = 2018;
```

order_month	total_orders
1	7269
2	6728
3	7211
4	6939
5	6873
6	6167
7	6292
8	6512
9	16
10	4

earliest_2018	latest_2018
2018-01-01 02:48:41	2018-10-17 17:30:18

```
# Ensure datetime format
orders['order_purchase_timestamp'] = pd.to_datetime(orders['order_purchase_timestamp'])

# Filter 2018 orders
orders_2018 = orders[orders['order_purchase_timestamp'].dt.year == 2018]

# Group by month
monthly_orders_2018 = (
    orders_2018
    .groupby(orders_2018['order_purchase_timestamp'].dt.month)['order_id']
    .nunique()
    .reset_index()
)

monthly_orders_2018.columns = ['order_month', 'total_orders']

display(monthly_orders_2018.sort_values('order_month'))
```

✓ 0.0s

order_month	total_orders
0	1
1	2
2	3
3	4
4	5
5	6
6	7
7	8
8	9
9	10

Interpretation

- The business showed **stable and high monthly order volume** throughout 2018 (Jan–Aug). Then strong decline.
- Analysis for full-year 2018 growth should be treated carefully due to incomplete data.

2. Find the average number of products per order, grouped by customer city

	customer_city	avg_products_per_order
2619	padre carvalho	7.0
907	celso ramos	6.5
756	candido godoi	6.0
1154	datas	6.0
2264	matias olimpio	5.0
...	...	...
1662	indiana	1.0
1663	indianapolis	1.0
1664	indiapora	1.0
1666	indiaroba	1.0
4109	zortea	1.0

4110 rows × 2 columns

Insights:

- Some smaller cities show very high averages (6–7 items per order).
- Majority of cities average close to **1 product per order**.
- Results likely influenced by low-order-volume cities.
- Bulk purchases in small cities inflate averages.

```
71 • SELECT
72     customer_city,
73     ROUND(AVG(order_item_count), 2) AS avg_products_per_order
74 FROM (
75     SELECT
76         o.order_id,
77         c.customer_city,
78         COUNT(oi.order_item_id) AS order_item_count
79     FROM orders o
80     JOIN order_items oi
81         ON o.order_id = oi.order_id
82     JOIN customers c
83         ON o.customer_id = c.customer_id
84     GROUP BY o.order_id, c.customer_city
85 ) t
86 GROUP BY customer_city
87 ORDER BY avg_products_per_order DESC;
```

customer_city	avg_products_per_order
padre carvalho	7.00
celso ramos	6.50
datas	6.00
candido godoi	6.00
matias olimpio	5.00
teixeira soares	4.00
morro de sao paulo	4.00
curralinho	4.00
picarra	4.00
cidelandia	4.00
inconfidentes	3.50
ipua	3.25
buriti	3.00
chapadao do lage...	3.00
inse hoiteux	3.00

3. Calculate the percentage of total revenue contributed by each product category

```
92 • SELECT
93     p.product_category,
94     ROUND(SUM(oi.price), 2) AS category_revenue,
95     ROUND(
96         SUM(oi.price) /
97         (SELECT SUM(price) FROM order_items) * 100,
98         2
99     ) AS revenue_percentage
100 FROM order_items oi
101 JOIN products p
102     ON oi.product_id = p.product_id
103 GROUP BY p.product_category
104 ORDER BY revenue_percentage DESC;
```

product_category	category_revenue	revenue_percentage
HEALTH BEAUTY	1258681.34	9.26
Watches present	1205005.68	8.87
bed table bath	1036988.68	7.63
sport leisure	988048.97	7.27
computer accessories	911954.32	6.71
Furniture Decoration	729762.49	5.37
Cool Stuff	635290.85	4.67
housewares	632248.66	4.65
automotive	592720.11	4.36
Garden tools	485256.46	3.57
toys	483946.60	3.56
babies	411764.89	3.03
perfumery	399124.87	2.94
telephony	323667.53	2.38
Furniture office	273960.70	2.02
stationary store	230943.23	1.70
PCs	222963.13	1.64
pet Shop	214315.41	1.58

Category	Revenue %
Health Beauty	9.26%
Watches present	8.87%
Bed table bath	7.63%
Sport leisure	7.27%
Computer accessories	6.71%

Insights:

- Top 5 categories contribute ~40% of total revenue.
- Health & Beauty alone drives 9.26% of total sales.
- Revenue is moderately concentrated but not monopolized.
- Orders shows diversified category performance.
- Long-tail categories contribute minimal revenue individually.

	product_category	category_revenue	revenue_percentage
30	HEALTH BEAUTY	1258681.34	9.26
45	Watches present	1205005.68	8.87
49	bed table bath	1036988.68	7.63
68	sport leisure	988048.97	7.27
53	computer accessories	911954.32	6.71
...	...	...	...
58	flowers	1110.04	0.01
32	House Comfort 2	760.27	0.01
2	Arts and Crafts	1814.01	0.01
62	insurance and services	283.29	0.00
18	Fashion Children's Clothing	569.85	0.00

73 rows × 3 columns

4. Identify the correlation between product price and the number of times a product has been purchased

```

116 • SELECT
117     oi.product_id,
118     p.product_category,
119     COUNT(oi.order_item_id) AS times_purchased,
120     ROUND(AVG(oi.price), 2) AS avg_price
121 FROM order_items oi
122 JOIN products p
123 ON oi.product_id = p.product_id
124 GROUP BY oi.product_id, p.product_category;
125

```

product_id	product_category	times_purchased	avg_price
8cab8abac59158715e0d70a36c807415	sport leisure	2	99.90
b50c950aba0dcead2c48032a690ce817	sport leisure	4	99.00
53b36df67ebb7c41585e8d54d6772e08	Watches present	323	116.67
5e21d5cab5d33e770d8150a4ee6117db	Watches present	12	49.00
d0fb1e667e989933a80444f93da833c0	housewares	1	37.90
601a360bd2a916ecef0e88de72a6531a	Cool Stuff	119	127.39
2f763ba79d9cd987b2034aac7ceffe06	electronics	1	469.00
e10758160da97891c2fdbc35f0f031d		2	56.00
31a2f42a87890f87d77daebdfabc182e	toys	42	146.46
ce130b350f529a1d6288f927da7404ea	bed table bath	4	64.90
a963ea4d494221de09b70f1b4a70edbd	computer access...	5	89.90
a8115d34a4b7b74388da609692162c37	computer access...	24	32.90
bf06a69bb27c58156391de458700f3e7	Watches present	3	38.00
464383c22e65860826cf78650fe0f4e4	Games consoles	1	89.99
4834f24983a4f056e5ae64265f723aaf	electronics	1	24.89
c6336fa91fbd87c359e44f5dca5a90ed	sport leisure	94	43.74
0e40b1ed4cfd3da1962ec91913e54ba8	electronics	15	13.65
121b9686b9929855d823981fc655a6fe	housewares	11	55.08

Price vs Purchase Frequency

- Correlation: **-0.03**
- No significant relationship between price and demand
- Expensive products are not necessarily purchased less
- Purchase behavior likely driven by:
 - Product relevance
 - Category demand
 - Brand perception
 - Necessity

Correlation between price and times purchased: -0.0328

	product_id	product category	times_purchased	avg_price
0	00066f42aeeb9f3007548bb9d3f33c38	perfumery	1	101.65
1	00088930e925c41fd95ebfe695fd2655	automotive	1	129.90
2	0009406fd7479715e4bef61dd91f2462	bed table bath	1	229.00
3	000b8f95fcb9e0096488278317764d19	housewares	2	58.90
4	000d9be29b5207b54e86aa1b1ac54872	Watches present	1	199.00

5. Calculate total revenue generated by each seller and rank them by revenue.

```
130 • SELECT
131     seller_id,
132     total_revenue,
133     RANK() OVER (ORDER BY total_revenue DESC) AS revenue_rank
134 FROM (
135     SELECT
136         oi.seller_id,
137         ROUND(SUM(oi.price + oi.freight_value), 2) AS total_revenue
138     FROM order_items oi
139     GROUP BY oi.seller_id
140 ) seller_revenue
141 ORDER BY revenue_rank;
142
```

seller_id	total_revenue	revenue_rank
4869f7a5dfa277a7dca6462dcf3b52b2	249640.70	1
7c67e1448b00f6e969d365cea6b010ab	239536.44	2
53243585a1d6dc2643021fd1853d8905	235856.68	3
4a3ca9315b744ce9f8e9374361493884	235539.96	4
fa1c13f2614d7b5c4749cbc52fecda94	204084.73	5
da8622b14eb17ae2831f4ac5b9dab84a	185192.32	6
7e93a43ef30c4f03f38b393420bc753a	182754.05	7
1025f0e2d44d7041d6cf58b6550e0bfa	172860.69	8
7a67c85e85bb2ce8582c35f2203ad736	162648.38	9
955fee9216a65b617aa5c0531780ce60	160602.68	10
6560211a19b47992c3666cc44a7e94c0	151265.77	11
1f50f920176fa81dab994f9023523100	142104.98	12
46dc3b2cc0980fb8ec44634e21d2718e	139909.69	13
a1043bafd471dff536d0c462352beb48	133745.25	14
cc419e0650a3c5ba77189a1882b7556a	129957.41	15
620c87c171fb2a6dd6e8bb4dec959fc6	128909.71	16
5dceca129747e92ff8ef7a997dc4f8ca	125816.92	17
7d13fca15225358621be4086e1eb0964	122261.02	18

	seller_id	total_revenue	revenue_rank
857	4869f7a5dfa277a7dca6462dcf3b52b2	249640.70	1.0
1535	7c67e1448b00f6e969d365cea6b010ab	239536.44	2.0
1013	53243585a1d6dc2643021fd1853d8905	235856.68	3.0
881	4a3ca9315b744ce9f8e9374361493884	235539.96	4.0
3024	fa1c13f2614d7b5c4749cbc52fecda94	204084.73	5.0
2643	da8622b14eb17ae2831f4ac5b9dab84a	185192.32	6.0
1560	7e93a43ef30c4f03f38b393420bc753a	182754.05	7.0
192	1025f0e2d44d7041d6cf58b6550e0bfa	172860.69	8.0
1505	7a67c85e85bb2ce8582c35f2203ad736	162648.38	9.0
1824	955fee9216a65b617aa5c0531780ce60	160602.68	10.0

Insights :

1. Seller Revenue Concentration

- Top seller generated **249,640.70**
- Top 5 sellers all exceed **200K revenue**
- Revenue distribution is highly skewed

2. Marketplace Structure

- Clear **leaderboard effect**
- Small number of sellers dominate total revenue

ADVANCED PROBLEMS

1. Calculate the moving average of order values for each customer over their order history.
 2. Calculate the cumulative sales per month for each year.
 3. Calculate the year-over-year growth rate of total sales.
 4. Calculate the retention rate of customers, defined as the percentage of customers who make another purchase within 6 months of their first purchase.
 5. Identify the top 3 customers who spent the most money in each year.
- 
- A solid orange horizontal bar spanning the width of the slide, located at the bottom.

1. Calculate the moving average of order values for each customer over their order history.

	customer_id	order_id	order_purchase_timestamp	order_value	moving_avg_order_value
▶	00012a2ce6f8dcda20d059ce98491703	5f79b5b0931d63f1a42989eb65b9da6e	2017-11-14 16:08:26	114.74	114.74
	000161a058600d5901f007fab4c27140	a44895d095d7e0702b6a162fa2dbeced	2017-07-16 09:40:32	67.41	67.41
	0001fd6190edaaf884bcaf3d49edf079	316a104623542e4d75189bb372bc5f8d	2017-02-28 11:06:43	195.42	195.42
	0002414f95344307404f0ace7a26f1d5	5825ce2e88d5346438686b0bba99e5ee	2017-08-16 13:09:20	179.35	179.35
	000379cdec625522490c315e70c7a9fb	0ab7fb08086d4af9141453c91878ed7a	2018-04-02 13:42:17	107.01	107.01
	0004164d20a9e969af783496f3408652	cd3558a10d854487b4f907e9b326a4fc	2017-04-12 08:35:12	71.80	71.80
	000419c5494106c306a97b5635748086	07f6c3baf9ac86865b60f640c4f923c6	2018-03-02 17:47:40	49.40	49.40
	00046a560d407e99b969756e0b10f282	8c3d752c5c02227878fae49aeaddbdf7	2017-12-18 11:08:30	166.59	166.59
	00050bf6e01e69d5c0fd612f1bcbf69c	fa906f338cee30a984d0945b3832e431	2017-09-17 16:04:44	85.23	85.23
	000598caf2ef4117407665ac33275130	9b961b894e797f63622137ff7eb1c1af	2018-08-11 12:14:35	1255.71	1255.71
	0005aefbb696d34b3424dccc0a0e9fd0	263ba12390d0fbc329dd16da8cd20f8	2018-06-20 09:46:53	147.33	147.33
	00062b33cb9f6fe976afdcff967ea74d	c208db5638f7f1cd04d185856852f864	2017-03-15 23:44:09	58.95	58.95
	00066ccbe787a588c52bd5ff404590e3	728416b0db65935dbf78a0cc03e8d6f8	2018-02-06 16:10:09	270.00	270.00
	00072d033fe2e59061ae5c3aff1a2be5	2346e1104a4b18a23e6dc6a87c2d1b8c	2017-09-01 09:24:39	106.97	106.97
	0009a69b72033b2d0ec8c69fc70ef768	048beca6ccda094fb80bbc704d7c493d	2017-04-28 13:36:30	173.60	173.60
	000bf8121c3412d3057d32371c5d3395	bc3e295306ee4d3eba91aca49b0bb539	2017-10-11 07:44:31	45.56	45.56
	000e943451fc2788ca6ac98a682f2f49	cfde948cbe426f37ac61e0f04215a5f3	2017-04-20 19:37:14	106.23	106.23
	000f17e290c26b28549908a04cfe36c1	cabfe1219e952ead9a914adde40efc42	2017-11-10 16:37:05	139.52	139.52

Insight :

- ❑ Each customer_id in the dataset corresponds to a single order.
- ❑ Therefore, no customer has multiple orders under the same customer_id.
- ❑ As a result:
 - Moving average = order value itself.
 - No spending trend can be observed at this level.

	order_id	order_value	customer_id	order_purchase_timestamp	moving_avg_order_value
36660	5f79b5b0931d63f1a42989eb65b9da6e	114.74	00012a2ce6f8dcda20d059ce98491703	2017-11-14 16:08:26	114.74
63005	a44895d095d7e0702b6a162fa2dbeced	67.41	000161a058600d5901f007fab4c27140	2017-07-16 09:40:32	67.41
18934	316a104623542e4d75189bb372bc5f8d	195.42	0001fd6190edaaf884bcaf3d49edf079	2017-02-28 11:06:43	195.42
33936	5825ce2e88d5346438686b0bba99e5ee	179.35	0002414f95344307404f0ace7a26f1d5	2017-08-16 13:09:20	179.35
4167	0ab7fb08086d4af9141453c91878ed7a	107.01	000379cdec625522490c315e70c7a9fb	2018-04-02 13:42:17	107.01
79083	cd3558a10d854487b4f907e9b326a4fc	71.80	0004164d20a9e969af783496f3408652	2017-04-12 08:35:12	71.80
3090	07f6c3baf9ac86865b60f640c4f923c6	49.40	000419c5494106c306a97b5635748086	2018-03-02 17:47:40	49.40
53821	8c3d752c5c02227878fae49aeaddbdf7	166.59	00046a560d407e99b969756e0b10f282	2017-12-18 11:08:30	166.59
96575	fa906f338cee30a984d0945b3832e431	85.23	00050bf6e01e69d5c0fd612f1bcbf69c	2017-09-17 16:04:44	85.23
59578	9b961b894e797f63622137ff7eb1c1af	1255.71	000598caf2ef4117407665ac33275130	2018-08-11 12:14:35	1255.71
14669	263ba12390d0fbc329dd16da8cd20f8	147.33	0005aefbb696d34b3424dccc0a0e9fd0	2018-06-20 09:46:53	147.33
74771	c208db5638f7f1cd04d185856852f864	58.95	00062b33cb9f6fe976afdcff967ea74d	2017-03-15 23:44:09	58.95
44127	728416b0db65935dbf78a0cc03e8d6f8	270.00	00066ccbe787a588c52bd5ff404590e3	2018-02-06 16:10:09	270.00
13562	2346e1104a4b18a23e6dc6a87c2d1b8c	106.97	00072d033fe2e59061ae5c3aff1a2be5	2017-09-01 09:24:39	106.97
1762	048beca6ccda094fb80bbc704d7c493d	173.60	0009a69b72033b2d0ec8c69fc70ef768	2017-04-28 13:36:30	173.60

2. Calculate cumulative sales per month for each year

	sales_year	sales_month	monthly_sales	cumulative_sales
0	2016	9	354.75	354.75
1	2016	10	56808.84	57163.59
2	2016	12	19.62	57183.21
3	2017	1	137188.49	137188.49
4	2017	2	286280.62	423469.11
5	2017	3	432048.59	855517.70
6	2017	4	412422.24	1267939.94
7	2017	5	586190.95	1854130.89
8	2017	6	502963.04	2357093.93
9	2017	7	584971.62	2942065.55
10	2017	8	668204.60	3610270.15
11	2017	9	720398.91	4330669.06
12	2017	10	769312.37	5099981.43
13	2017	11	1179143.77	6279125.20
14	2017	12	863547.23	7142672.43
15	2018	1	1107301.89	1107301.89
16	2018	2	986908.96	2094210.85
17	2018	3	1155126.82	3249337.67
18	2018	4	1159698.04	4409035.71
19	2018	5	1149781.82	5558817.53
20	2018	6	1022677.11	6581494.64
21	2018	7	1058728.03	7640222.67
22	2018	8	1003308.47	8643531.14
23	2018	9	166.46	8643697.60

Revenue Growth Pattern:

➤ 2016

- Minimal activity (partial year)
- Revenue starts from September
- Total cumulative revenue ≈ **57K**

➤ 2017

- Strong and consistent growth
- Revenue increases steadily month over month
- Year-end cumulative revenue approx. **7.14M**
- Clear business expansion phase

➤ 2018

- Very strong start
- By August 2018, cumulative revenue already exceeds **8.64M**
- Surpasses entire 2017 revenue before year completion
- September shows extremely low activity (likely incomplete dataset)

```
175
176 -- Advanced Problem 2
177 -- Cumulative monthly sales per year
178
179 SELECT
```

	sales_year	sales_month	monthly_sales	cumulative_sales
▶	2016	9	354.75	354.75
	2016	10	56808.84	57163.59
	2016	12	19.62	57183.21
	2017	1	137188.49	137188.49
	2017	2	286280.62	423469.11
	2017	3	432048.59	855517.70
	2017	4	412422.24	1267939.94
	2017	5	586190.95	1854130.89
	2017	6	502963.04	2357093.93
	2017	7	584971.62	2942065.55
	2017	8	668204.60	3610270.15
	2017	9	720398.91	4330669.06
	2017	10	769312.37	5099981.43
	2017	11	1179143.77	6279125.20
	2017	12	863547.23	7142672.43
	2018	1	1107301.89	1107301.89
	2018	2	986908.96	2094210.85
	2018	3	1155126.82	3249337.67
	2018	4	1159698.04	4409035.71
	2018	5	1149781.82	5558817.53
	2018	6	1022677.11	6581494.64
	2018	7	1058728.03	7640222.67
	2018	8	1003308.47	8643531.14
	2018	9	166.46	8643697.60

3. Year-over-Year Growth Rate of Total Sales

```
)  
  
# Calculate YoY growth  
yearly_sales['yoy_growth_percentage'] = (  
    yearly_sales['total_sales'].pct_change() * 100  
).round(2)  
  
display(yearly_sales)
```

36] ✓ 0.1s

C:\Users\WIVAS\AppData\Local\Temp\ipykernel_21164\1615890193.py:17: FutureWarning: DataFrame.apply(lambda x: (x['price'] + x['freight_value']).sum())

	sales_year	total_sales	yoy_growth_percentage
0	2016	57183.21	NaN
1	2017	7142672.43	12390.86
2	2018	8643697.60	21.01

Insights:

- Massive growth from **2016 to 2017 (12,390%)**, driven by 2016 being a partial year.
- Strong and realistic growth from **2017 to 2018 (21.01%)**, indicating healthy business expansion.
- 2018 shows sustained scaling rather than one-time spike.
- Business transitioned from early-stage (2016) to growth phase (2017–2018).

```
206 • SELECT  
207     sales_year,  
208     total_sales,  
209     ROUND(  
210         (  
211             total_sales - LAG(total_sales) OVER (ORDER BY sales_year)  
212         )  
213         / LAG(total_sales) OVER (ORDER BY sales_year) * 100,  
214         2  
215     ) AS yoy_growth_percentage  
216 FROM (  
217     SELECT  
218         YEAR(o.order_purchase_timestamp) AS sales_year,  
219         ROUND(SUM(oi.price + oi.freight_value), 2) AS total_sales  
220     FROM orders o  
221     JOIN order_items oi  
222         ON o.order_id = oi.order_id  
223     GROUP BY sales_year  
224 ) yearly_sales  
225 ORDER BY sales_year;
```

Result Grid

	sales_year	total_sales	yoy_growth_percentage
▶	2016	57183.21	NULL
	2017	7142672.43	12390.86
	2018	8643697.60	21.01

4. Calculate the retention rate of customers, defined as the percentage of customers who make another purchase within 6 months of their first purchase.

Insights:

- Customer retention rate is **0%**.
- Dataset shows **one order per customer** (no repeat purchases).
- Retention analysis is not meaningful due to dataset structure.
- Indicates either:
 - Early-stage customer base
 - Or dataset limitation (no repeat transaction history)

```
229 • SELECT
230     ROUND(
231         (retained_customers / total_customers) * 100,
232         2
233     ) AS retention_rate_percentage
234 FROM (
235     SELECT
236         (SELECT COUNT(DISTINCT customer_id) FROM orders) AS total_customers,
237         (
238             SELECT COUNT(DISTINCT o.customer_id)
239             FROM orders o
240             JOIN (
241                 SELECT
242                     customer_id,
243                     MIN(order_purchase_timestamp) AS first_purchase_date
244                 FROM orders
245                 GROUP BY customer_id
246             ) fp
247             ON o.customer_id = fp.customer_id
248             WHERE o.order_purchase_timestamp > fp.first_purchase_date
249                 AND o.order_purchase_timestamp <= DATE_ADD(fp.first_purchase_date, INTERVAL 6 MONTH)
250         ) AS retained_customers
251     ) t;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

retention_rate_percentage
0.00

```
print("Total Customers:", total_customers)
print("Retained Customers (within 6 months):", retained_customers)
print("Retention Rate:", retention_rate, "%")
```

✓ 0.2s

```
Total Customers: 99441
Retained Customers (within 6 months): 0
Retention Rate: 0.0 %
```

5. The top 3 customers who spent the most money in each year.

```
.apply(lambda x: (x['price'] + x['freight_value']).sum())
```

	sales_year	customer_id	total_spent	spend_rank
213	2016	a9dc96b027d1252bbac0a9b72d837fc6	1423.55	1.0
37	2016	1d34ed25963d5aae4cf3d7f3a4cda173	1400.74	2.0
80	2016	4a06381959b6670756de02e07b83815f	1227.78	3.0
4156	2017	1617b1357756262bfa56ab541c47bc16	13664.08	1.0
35059	2017	c6e2731c5b391845f6800c97401a43a9	6929.31	2.0
11418	2017	3fd6777bbce08a352fddd04e4a7cc8f6	6726.66	3.0
94592	2018	ec5b2ba62e574342386871631fafd3fc	7274.88	1.0
96320	2018	f48d464a0baaea338cb25f816991ab1f	6922.21	2.0
92125	2018	e0a2412720e9ea4f26c1ac985f6a7358	4809.44	3.0

Top 3 Customers by Year:

➤ 2016

Highest spend ≈ 1,423

➤ 2017

Highest spend ≈ 13,664

Significant jump vs 2016 → strong revenue scaling

➤ 2018

Highest spend ≈ 7,275

More balanced distribution among top 3

No extreme outlier like 2017

```

256 • SELECT
257     sales_year,
258     customer_id,
259     total_spent,
260     spend_rank
261 FROM (
262     SELECT
263         YEAR(o.order_purchase_timestamp) AS sales_year,
264         o.customer_id,
265         ROUND(SUM(oi.price + oi.freight_value), 2) AS total_spent,
266         RANK() OVER (
267             PARTITION BY YEAR(o.order_purchase_timestamp)
268             ORDER BY SUM(oi.price + oi.freight_value) DESC
269         ) AS spend_rank
270     FROM orders o
271     JOIN order_items oi
272         ON o.order_id = oi.order_id
273     GROUP BY sales_year, o.customer_id
274 ) ranked_customers
275 WHERE spend_rank <= 3
276 ORDER BY sales_year, spend_rank;

```

sales_year	customer_id	total_spent	spend_rank
2016	a9dc96b027d1252bbac0a9b72d837fc6	1423.55	1
2016	1d34ed25963d5aae4cf3d7f3a4cda173	1400.74	2
2016	4a06381959b6670756de02e07b83815f	1227.78	3
2017	1617b1357756262bfa56ab541c47bc16	13664.08	1
2017	c6e2731c5b391845f6800c97401a43a9	6929.31	2
2017	3fd6777bbce08a352fddd04e4a7cc8f6	6726.66	3
2018	ec5b2ba62e574342386871631fafd3fc	7274.88	1
2018	f48d464a0baaea338cb25f816991ab1f	6922.21	2
2018	e0a2412720e9ea4f26c1ac985f6a7358	4809.44	3